ProBuild Operations Platform (v2) — Test Strategy

Field	Value
Document	Test Strategy
System	Orchestration-based Camunda 8 process platform (7 processes, 4 DMN, 24 workers)

1. Objectives

1. Every gateway branch in `Journey_Customer` reaches its expected end event (served / cancelled) without the process stalling or raising an incident.
2. Each **call activity** invokes its capability process and returns the agreed output contract (`Cap_Sales`, `Cap_Warehouse`, `Cap_Loyalty`, `Cap_Hire`).
3. Cross-pool **message correlation** holds end-to-end: `finance.requested` / `finance.settled` (Customer ↔ FinTrust) and `service.requested` / `service.completed` (Customer ↔ FixPro), all keyed on `journeyId`.
4. Each **DMN decision table** is evaluated and drives the process the way its rules dictate (finance APPROVE/DECLINE, loyalty band, hire tier, maintenance split).
5. Every automatic step leaves the correct **H2 side-effect**.

2. Scope

In scope: all 3 customer journeys and every gateway branch; all 7 processes; the 4 message channels; the 4 DMN tables; H2 persistence; form-level validation.

Out of scope: load/performance; security (handled by the Camunda platform); the timer-triggered weekly FixPro sweep is deployed and unit-exercised but not scheduled-tested (it fires on a 7-day cycle); UI regression of Operate/Tasklist (third-party).

The approach follows established BPMN testing and method-and-style guidance (Silver, 2011; Freund and Rücker, 2019) and Camunda's process-testing recommendations (Camunda, 2024). Full list: `docs/references.md`.

3. Approach — black-box, API-driven, end-to-end

Testing is driven through the engine's **v2 REST API**, mirroring how an operator would use Tasklist/Operate but making the suite repeatable. For each case: start a `Journey_Customer` instance, complete user tasks in order (including tasks that surface in the FinTrust and FixPro pools), let service tasks and DMN business-rule tasks run, then assert on (a) the end event reached, (b) the state of called capability and partner instances, and (c) H2 rows.

Rationale: this validates the full stack — orchestrator → call activity → capability worker → DMN → message → partner → database — in one execution, and it is the only way to prove the multi-pool

message correlation actually closes the loop.

4. Pass/fail criteria

A case **PASSES** when: the journey reaches the expected end event; all called capability and partner instances are `COMPLETED`; the expected H2 rows exist with correct values; and no incident is raised. Otherwise it **FAILS** and the deviation is recorded in the report.

5. Form validation

Form-level validation is enforced by Camunda Tasklist at the point of entry (required fields, numeric `min` / `max`, `email` /pattern checks, and conditional finance fields on the payment form). A user task whose form fails validation cannot be completed, so a worker never receives out-of-range input. Read-only summary forms (quote, deposit, confirmation) present server-computed values the operator cannot tamper with.

6. Risk analysis

Risk	Likelihood	Impact	Mitigation
Message not correlated across a pool boundary → catch event waits forever	Medium	High	`journeyId` correlation key asserted in TC-BUY-02 and TC-HIRE-01; both message round-trips exercised
Call activity output contract drifts from orchestrator expectation	Medium	High	Each capability returns an explicit outcome variable (`salesOutcome` , `hireOutcome` , `returnOutcome`) gated in the orchestrator
A worker overwrites a variable used downstream (silent £0 pricing)	Medium	High	Financial values asserted directly in H2, not just end-event arrival
Missing form → `FORM_NOT_FOUND` incident blocks a user task	Low	High	All 17 forms deployed with the processes; verified at startup
DMN rule gap leaves a decision unmatched	Low	Medium	Every table has a catch-all/ `FIRST` -policy fallback rule

7. Coverage

10 journey paths cover: card new/renewal; buy in-stock (card & finance-approved & finance-declined), out-of-stock, quote-rejected; hire with-maintenance, clean-return, unavailable. This exercises every gateway branch, all four DMN tables, both partner message round-trips, and both terminal end events (served / cancelled).